

Пул потоков на C++: постановка задачи и план экспериментов

1. Постановка задачи

В современных многопоточных приложениях часто возникает необходимость выполнять большое число относительно независимых вычислительных задач. Наивный подход — создавать новый поток под каждую задачу — сопряжён с существенными накладными расходами: создание и уничтожение потока в ОС занимает время, а чрезмерное число потоков приводит к деградации производительности из-за конкуренции за процессорное время и кэш.

Требуется реализовать на C++ структуру данных «пул потоков» (thread pool), которая:

- создаёт фиксированное число рабочих потоков при инициализации и поддерживает их в течение всего времени жизни объекта;
- принимает произвольные вызываемые объекты (функции, лямбды, указатели на функции с аргументами) и ставит их в очередь на выполнение;
- назначает каждой поставленной задаче уникальный целочисленный идентификатор (`task_id`), возвращаемый вызывающей стороне;
- предоставляет блокирующее ожидание завершения конкретной задачи по её `task_id` (`Wait`);
- предоставляет блокирующее ожидание завершения **всех** поставленных задач (`WaitAll`);
- позволяет неблокирующим образом проверить, завершена ли задача (`IsSubmitted`);
- корректно завершает работу при разрушении объекта: все запущенные задачи доводятся до конца, рабочие потоки останавливаются.

Интерфейс

```
namespace threadpool {

class ThreadPool {
public:
    explicit ThreadPool(size_t threads_count);

    // Ставит задачу в очередь; возвращает уникальный идентификатор
    template <typename Func, typename... Args>
    std::int64_t AddTask(const Func& task_func, Args&&... args);

    // Блокирует вызывающий поток до завершения задачи task_id
    void Wait(std::int64_t task_id);

    // Блокирует вызывающий поток до завершения всех задач
    void WaitAll();

    // Неблокирующая проверка завершённости задачи
    bool IsSubmitted(std::int64_t task_id);

    ~ThreadPool();
};

} // namespace threadpool
```

Ограничения и требования к реализации

- Язык: C++17 и выше.
- Сборочная система: CMake (версия ≥ 3.20).
- Запрещено использование сторонних библиотек для управления потоками (только STL: `<thread>`, `<mutex>`, `<condition_variable>`, `<future>`, `<atomic>`).
- Реализация не должна содержать гонок данных, дедлоков и утечек потоков.

2. Цель работы

Разработать потокобезопасную реализацию пула потоков, обеспечивающую:

1. **Переиспользование потоков** — исключить накладные расходы на создание/уничтожение потоков при обработке серии задач.
2. **Управление параллелизмом** — ограничить число одновременно работающих потоков заданной константой `threads_count`, предотвращая перегрузку системы.
3. **Гибкость API** — поддерживать постановку задач любой сигнатуры через шаблонный метод `AddTask`.
4. **Детерминированное завершение** — гарантировать, что все поставленные задачи будут выполнены до уничтожения объекта `ThreadPool`.
5. **Мониторинг выполнения** — дать пользователю возможность как синхронно ожидать результатов (`Wait`, `WaitAll`), так и асинхронно опрашивать состояние задачи (`IsSubmitted`).

3. Планируемые эксперименты

Эксперимент 1 — Базовая корректность

Цель: убедиться, что задачи выполняются и результаты записываются корректно.

Метод: поставить в пул `$N$` задач вычисления суммы элементов непересекающихся массивов, дождаться завершения через `WaitAll`, сравнить результаты с эталоном (однопоточный расчёт).

Ожидаемый результат: все суммы совпадают с эталонными при любом значении `threads_count` от 1 до `$N$`.

Эксперимент 2 — Корректность Wait и IsSubmitted

Цель: проверить, что `Wait(id)` возвращает управление только после завершения задачи с данным `id`, а `IsSubmitted` возвращает `true` именно и только для завершённых задач.

Метод:

- поставить задачу с искусственной задержкой (`std::this_thread::sleep_for`);
- сразу после `AddTask` вызвать `IsSubmitted` — ожидается `false`;
- вызвать `Wait(id)` и сразу после него повторить `IsSubmitted` — ожидается `true`.

Ожидаемый результат: состояния задачи переходят `pending` → `submitted` ровно один раз, не раньше фактического завершения.

Эксперимент 3 — Нагрузочный тест (throughput)

Цель: измерить пропускную способность пула в зависимости от числа потоков.

Метод: поставить 10000 коротких задач (инкремент атомарного счётчика), зафиксировать время от первого `AddTask` до возврата из `WaitAll`.

Варьируемый параметр: `threads_count` ∈ {1, 2, 4, 8, 16}.

Метрика: задач/сек; построить график зависимости пропускной способности от числа потоков.

Ожидаемый результат: рост до числа логических ядер, затем — насыщение или деградация из-за накладных расходов на синхронизацию.

Эксперимент 4 — Сравнение с наивным подходом (spawn-per-task)

Цель: количественно обосновать целесообразность использования пула.

Метод: для фиксированного числа задач (`N = 1000`) сравнить полное время выполнения:

- пул потоков с `threads_count` = числу логических ядер;
- создание нового `std::thread` на каждую задачу с последующим `join`.

Метрика: среднее время по 10 прогонам, 95-й перцентиль.

Ожидаемый результат: пул быстрее за счёт отсутствия накладных расходов на создание потоков.

Эксперимент 5 — Тест на гонки (thread sanitizer)

Цель: верифицировать отсутствие гонок данных в реализации.

Метод: компилировать с флагом `-fsanitize=thread` и запускать все предыдущие сценарии. Дополнительно: несколько потоков-«производителей» одновременно вызывают `AddTask`, пока основной поток вызывает `WaitAll`.

Ожидаемый результат: TSan не сообщает ни об одной гонке.

Эксперимент 6 — Корректность деструктора при наличии задач в очереди

Цель: убедиться, что уничтожение пула не приводит к потере задач или зависанию.

Метод:

- создать пул с 2 потоками;
- поставить 100 задач, каждая из которых увеличивает глобальный счётчик;
- немедленно разрушить объект `ThreadPool` (выход из области видимости);
- проверить, что счётчик равен 100.

Ожидаемый результат: все задачи выполнены; программа завершается без зависания.

Эксперимент 7 — Масштабирование на CPU-bound нагрузке

Цель: оценить эффективность параллелизации вычислительно-ёмких задач.

Метод: разбить вычисление числа π методом Монте-Карло (10^8 итераций) на равные части и распределить по потокам пула. Сравнить время с однопоточным вариантом.

Варьируемый параметр: `threads_count` $\in \{1, 2, 4, 8\}$.

Метрика: ускорение (`speedup`) = $T1 / Tn$.

Ожидаемый результат: близкое к линейному ускорение вплоть до числа физических ядер.

4. Ожидаемые результаты и критерии успеха

Критерий	Условие успеха
Корректность (экс. 1–2)	Все результаты совпадают с эталоном; состояния задач переходят верно
Отсутствие гонок (экс. 5)	TSan: 0 предупреждений
Производительность (экс. 3–4)	Пул быстрее <code>spawn-per-task</code> минимум в 2× на $N = 1000$
Масштабирование (экс. 7)	<code>Speedup</code> $\geq 0.8\times$ от числа потоков на физических ядрах
Корректность завершения (экс. 6)	Счётчик = N ; нет зависания; нет утечек (<code>valgrind clean</code>)